Chapter 14

Latency Optimization

After reading this chapter, you will be able to:

- Measure serving latency correctly: report percentiles from histograms at a stated load and measurement point, warm up before measuring, repeat runs and report confidence intervals, and generate load in a way that avoids coordinated omission.
- Attribute end-to-end tail latency to pipeline stages with traces, distinguishing service time from waiting time and critical-path work from parallel slack, and conditioning on slow requests instead of reading per-stage percentiles.
- Explain, with Little's law, a simple queueing model, and the fan-out argument, why tail latency rises sharply with utilization, variability, and the number of parallel calls, and when hedged requests help.
- Select and sequence optimization levers (precomputation, caching, candidate budgets, approximate search operating points, batching, graph optimization, quantization, distillation, and hardware choice) using measured contribution, expected gain, quality risk, and cost.
- Derive per-stage latency allowances from measured distributions, propagate deadlines, set timeouts from downstream percentiles, and design a degradation ladder whose quality cost is measured before launch.
- Verify that an optimization preserved recommendation quality with score, rank, metric, and slice parity checks and a performance regression benchmark, before handing the change to the release gates of Chapter 15.

14.1 When Milliseconds Change Outcomes

In 2009, Google ran a series of experiments that few product teams would volunteer for: it made search slower on purpose. One group of users had their results held on the server for an extra 100 to 400 milliseconds, while a second group served as a control. Over four to six weeks, the delayed users ran fewer searches per day, by 0.2% at 100 ms and by 0.6% at 400 ms. The effect grew during the experiments, with the second half of the exposure period worse than the first, and it outlasted the delay: over the five weeks after the 400 ms delay was removed, those users still searched less than the control group, by 0.21% on average (Brutlag, 2009).

Microsoft's Bing ran the same kind of test, which this chapter calls a slowdown experiment: deliberately adding delay for a randomly assigned group of users and measuring what changes. For two weeks, one group of users received results 100 ms later than normal and a second group 250 ms later. The team estimated that every 100 ms of speedup was worth 0.6% of revenue (Kohavi et al., 2013).

Both studies add delay deliberately. Comparing fast and slow requests in ordinary logs confounds latency with whatever makes requests slow in the first place, such as heavy users, complex queries, and poor networks, and randomizing an injected delay removes that confounding. The estimate describes

the cost of a small slowdown at the current operating point, and reading it as the value of a small speedup assumes the effect is roughly linear near that point, an assumption Kohavi et al. (2020) make explicit. They also recommend tracking latency as a guardrail metric in online experiments, because a model that improves its target metric while slowing the service can lose through the second effect what it gained through the first. Small effects also need large samples: Kohavi et al. (2013) suggest that a reported finding of no impact from 200 ms of delay more plausibly reflected an experiment without the statistical power to detect it.

Delays below what users can perceive might seem harmless, and Arapakis et al. (2014) tested that idea with Yahoo search. In a controlled user study, participants could not reliably tell when delays under 500 ms had been added, delays above 1,000 ms were likely to be noticed, and participants used to a fast system noticed added delay more often than those used to a slow one. In large-scale search logs, however, users clicked more often on the faster of two otherwise identical result pages. A delay too small to notice can still change what people do.

All of this evidence comes from web search, where a user types a query and waits for a page. Outside search, Chapter 13 (Section 13.1) cited a randomized trial at Booking.com in which an increase of about 30% in latency cost more than 0.5% in conversion rate (Bernardi et al., 2019). Public, controlled evidence on how latency affects the users of a recommender is scarce, and each figure above describes one study under its own conditions; none is a constant to carry into another product. A home feed may be fetched before the user opens the app, and a carousel may load below the visible screen, so users do not always wait on a recommender the way they wait on a results page. The direction of the effect transfers more safely than its size, and the size has to be measured for each surface with the experimental methods of Chapter 16.

Recommendation adds a second reason to care about speed: latency decides which models can run at all. Every ranking model must finish inside the time the request allows, so a team that cannot make serving faster cannot ship a larger model, however much better it scores offline. Pinterest described this constraint lifting when it moved recommendation model serving from CPUs to GPUs. Serving its newer, larger models on CPUs would have raised cost and latency by up to 40 times. After a series of serving optimizations, GPUs served a model 77 times larger than before at 30% lower latency and neutral cost, and the team reported a 16% gain in Homefeed engagement from such larger models at neutral infrastructure cost (Eksombatchai and Zhang, 2022).

Chapter 13 ended with a working pipeline, a Locust file for load-testing it (Section 13.11.3), and a list of optimizations deferred to this chapter (Section 13.12). This chapter starts from that point and argues that latency work is an empirical discipline. A team measures the distribution of response times correctly, attributes the slow requests to the stages that caused them, chooses levers by measured contribution, expected gain, quality risk, and cost, and then proves that the faster system still recommends what the slower one did. Rules of thumb about which stage is slow, which batch size is best, or whether a GPU pays off fail often enough that the chapter treats each as a hypothesis to test.

14.2 Latency Is a Distribution

Chapter 2's example design answers included a 95th-percentile latency budget of 200 ms, and Chapter 13 described latency with percentiles (Section 13.1.1). This section makes that vocabulary precise, because most latency mistakes start with a number whose definition was never written down.

14.2.1 What a Latency Number Means

Latency is the elapsed time between two named events. "Recommendation latency" means nothing until the events are named: the request's first byte reaching the service and the last response byte leaving it, or a tap on the screen and the shelf rendering. Each pair gives a different number, and comparing numbers from different pairs produces arguments about differences that are not there.

Within a service, the elapsed time splits into two parts. Service time is the time spent doing the request's work: computing embeddings, searching an index, scoring candidates. Waiting time is the time a request spends ready to proceed but held back, in a connection queue, a thread pool queue, an event loop that is busy with another request, or a batch that has not yet filled. Response time is their sum. The two respond to different remedies: a faster model reduces service time directly and waiting time only by freeing capacity, and Section 14.5 shows that waiting time can grow without limit as load rises.

14.2.2 Percentiles and the Tail

Response times vary from request to request, so a single number describes a distribution only partially. The p-th percentile is the value below which p percent of observations fall. The notation p50, p95, p99, and p99.9 denotes the 50th, 95th, 99th, and 99.9th percentiles; p50 is the median. Tail latency refers to the high percentiles, conventionally p99 and above, and the slow requests that produce them.

Serving latencies are rarely symmetric. Most requests take a typical path, while a minority hit a cache miss, a cold model, a garbage-collection pause, or a queue behind a burst of traffic, and they take much longer. The result is a distribution with a long right tail, and often with several humps, one for each path: cache hits in one cluster, misses in another. Figure 14.1 shows a right-skewed shape of this kind, in which the mean, pulled toward the tail, describes no request in particular.

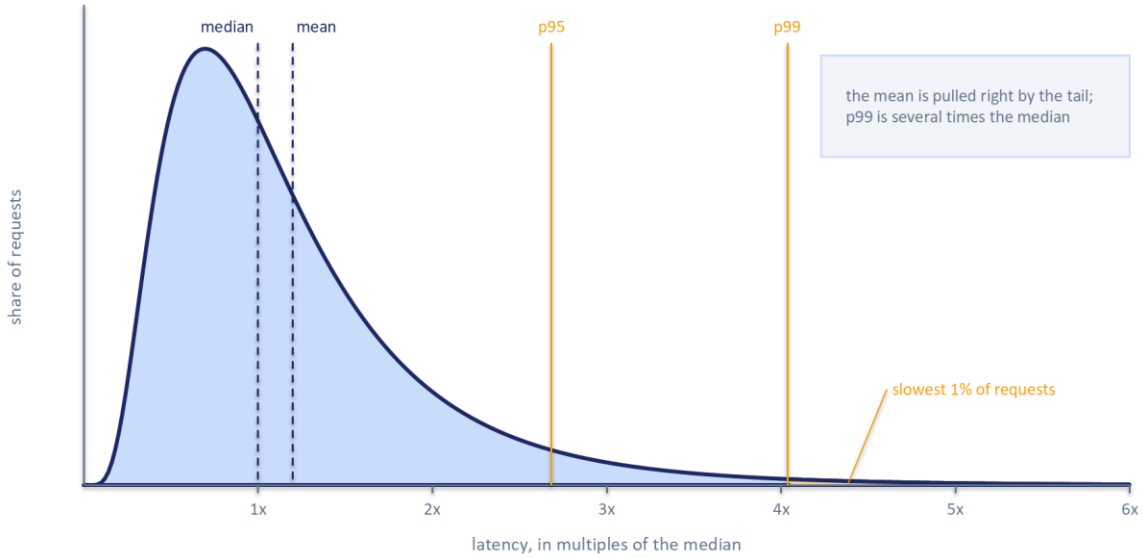


Figure 14.1: An illustrative right-skewed latency distribution, drawn as a lognormal shape in multiples of the median. The mean lies to the right of the median because the tail pulls it, and the 95th and 99th percentiles sit several times further out; the labeled region beyond p99 holds the slowest 1% of requests.

Why care about the slowest one percent? Because users do not issue one request. A session that loads a home page, scrolls, opens an item, and returns may trigger dozens of recommendation calls. If each call

independently exceeds the p99 with probability 0.01, the probability that a session of n calls contains at least one such call is

$$1 - 0.99^{n}.$$

For 10 calls that is about 10%, for 50 calls about 39%, and for 100 calls about 63%. The p99 is the experience of many users at some point in their session. Correlated slow periods make the true share somewhat lower, by concentrating slow calls in fewer sessions, and the users caught in them see several slow calls in a row. The same arithmetic applies inside a single request that fans out to many parallel backends, where the slowest of many component calls sets the pace; Section 14.5 develops that case following Dean and Barroso (2013).

14.2.3 Aggregating Percentiles Correctly

Percentiles do not combine the way means do. The average of the p99 measured on ten hosts is not the p99 of the fleet; if one host is slow, the fleet's p99 may be that host's p99, which averaging dilutes. The same holds across time windows: averaging five one-minute p99 values does not produce the five-minute p99. Stage percentiles also do not add to the end-to-end percentile, because the slowest requests for one stage are rarely the slowest for another, and the p99 of a sum is usually smaller than the sum of the p99s. A system that stores only precomputed percentiles has therefore thrown away the information needed to answer the next question.

The remedy is to record distributions as histograms, with counts of observations in latency buckets, and to merge histograms by adding the counts before computing any percentile. Tene (2012) describes HdrHistogram, a histogram design that keeps a fixed relative precision across a wide range of values, and argues that a latency requirement is incomplete until it states percentiles and a maximum. Histograms from many hosts and windows can be merged exactly, so the fleet p99 and the five-minute p99 are both available from the same records.

14.2.4 Measurement Points and Latency Targets

A latency number also depends on where it is measured, and Table 14.1 lists the common measurement points along a request's path. Each point includes some sources of delay and excludes others. The Chapter 13 orchestrator records each stage's latency in one structured log record per request (Section 13.8.1), which is a stage span in the table's terms. Those timings capture each stage's network round trip and the downstream service's work, and they miss everything outside the handler: time queued before the server began the request, the serialization of the response, and the network path back to the client.

Table 14.1: Where latency is measured.

Measurement point	Includes	Excludes	Typical use	Caveat
Client device	Network both ways, server time, rendering if timed to display	Nothing on the server side	What the user experiences	Noisy; device and network vary widely

Measurement point	Includes	Excludes	Typical use	Caveat
Edge or load balancer	Server time, queueing inside the service, response transfer	Client network and rendering	Fleet-wide service health	Mixes all endpoints unless split by route
Service ingress, before queueing	Queueing and handling within the service	Network outside the service	Capacity planning	Requires instrumentation at accept time
Handler start, after queueing	Request work and downstream calls	Waiting before the handler ran	Code-level profiling	Hides queueing, which dominates under load
Stage span	One stage's call, including its network round trip	Other stages and overhead between them	Attribution to stages	Spans do not add up to the request total

Chapter 13 introduced the latency target with two parts, a percentile and a threshold (Section 13.1.1). With the terms above in place, a target that can be tested needs three more, so this chapter uses the phrase for a statement with five parts: a percentile, a threshold, a measurement point, a load level, and a time window. An example is "the 99th percentile of orchestrator-measured response time stays below 200 ms at the expected peak request rate, evaluated over one-hour windows". Leaving out any part makes the target unfalsifiable: a p99 without a load level can be met by testing at low traffic, and a p99 without a measurement point can be met by timing a narrower interval. Chapter 15 (Section 15.2.2) formalizes the same idea as a service level indicator and objective with an error budget for production monitoring. This chapter uses latency targets to decide what to optimize and when to stop.

14.3 Measuring Without Fooling Yourself

An optimization is only as credible as the measurement that justified it. Serving benchmarks fail in a few recurring ways: they time a system that has not warmed up, report one run as if it were a population, time a component in isolation and assume the result transfers, or generate load in a way that hides the stalls they were meant to find.

14.3.1 Warmup, Repetition, and Noise

The first requests to a freshly started service are slower than the rest, for reasons that have nothing to do with steady-state performance. Models may load lazily on first use, memory allocators request pages from the operating system on first allocation, index files are read from disk until the operating system's page cache holds them, inference runtimes create sessions and select kernels on the first call, and connection pools start empty. These cold-start effects matter for deployments and autoscaling (Section 14.9 returns to them), and they contaminate a measurement of steady state. A benchmark therefore runs a warmup period, discards it, and measures only once the latency distribution has stopped drifting.

Even at steady state, two runs of the same benchmark on the same machine rarely agree. Georges et al. (2007) showed for managed-runtime benchmarks that common reporting habits, such as quoting a

single run or the best of several, could lead to misleading or incorrect conclusions, and recommended repeating independent runs and comparing alternatives with confidence intervals. The method transfers directly to serving benchmarks: run each configuration several times, restarting the process between runs if start-up state could differ, and report the percentile of interest with an interval across runs. If the intervals for two configurations overlap substantially, the benchmark has not shown that one is faster.

The machine contributes its own noise. Laptops change CPU frequency with power plans, temperature, and whether they are plugged in. Background processes such as file synchronization and indexing compete for cores and disk. Thread oversubscription is a quieter source: when a numerical library, an inference runtime, and a web server each start a thread pool sized to the number of cores, the process runs several times more threads than cores, and latency becomes erratic. A cloud virtual machine removes some of these sources and adds others, such as neighbors on the same physical host. Pinning thread counts, recording the machine's configuration with each result, and repeating runs make the noise visible. A load generator in the same process as the service shares its pauses: a garbage-collection pause stops both, and the requests due during it then arrive in a burst. Generating load from a separate process, or recording how late each request was sent, separates the generator's delays from the service's.

14.3.2 Microbenchmarks and End-to-End Benchmarks

A microbenchmark times one component in a loop: a model's forward pass on a fixed batch, or an index search on a fixed query set. It is fast and precise, and suits comparing two versions of one component. An end-to-end benchmark sends requests to the whole service under load and times them at a stated measurement point. It is slower and noisier, and it is the only way to see interactions: a model that is faster in isolation may contend for cores with feature fetching, and a larger batch may look efficient in a loop while adding waiting time under real arrival patterns. A credible optimization is usually shown twice, once in a microbenchmark that isolates the change and once end to end at the load level named in the latency target.

14.3.3 Closed-Loop and Open-Loop Load

Load generators come in two kinds. A closed-loop generator simulates a fixed number of clients, each of which sends a request, waits for the response, optionally pauses, and then sends the next. An open-loop generator sends requests on a schedule, such as a fixed rate or a random arrival process, whether or not earlier requests have completed. Real traffic from a large user population behaves like an open loop: users do not wait for each other's requests to finish before tapping their screens.

Locust, which Chapter 13 sets up to ramp simulated users in steps (Section 13.11.3), is closed-loop by design. Each simulated user runs a task, and the `wait_time` it declares is applied after the task completes. The throughput-oriented wait functions, `constant_throughput` and `constant_pacing`, adjust that pause to aim for a rate, and `poisson` randomizes it, but all of them can only slow a user down, never send a request while the previous one is still outstanding (Locust, 2026). Percentiles from such a test therefore describe the service under a fixed number of clients, each waiting for its previous response, a legitimate question that differs from how the service behaves at a given arrival rate.

The difference becomes decisive when the server stalls. Figure 14.2 shows why: a closed-loop client blocked on its one outstanding request sends nothing during the stall and records a single slow sample, while the requests real users would have sent, each waiting for the stall to end, never exist in the data. Tene (2012) named this coordinated omission, because the generator coordinates with the server and omits exactly the samples that describe the stall. Consider a 100-second run meant to send 100 requests

per second against a service that freezes once for one second. A closed-loop client records one request that took a second, one sample in about ten thousand. An open-loop schedule would have issued about 100 requests during the freeze, each waiting between almost nothing and nearly a second, so about one percent of the run's requests would carry the delay and the p99.9 would sit near nine-tenths of a second. The closed-loop record shows a p99.9 close to normal.



Figure 14.2: How closed-loop load hides a server stall. The closed-loop generator (top) waits for each response, so it sends nothing during the stall and records a single slow sample. The open-loop generator (bottom) sends requests at their intended times, and every request issued during the stall waits for it to end before being served, so each delayed request appears in the data. Schematic.

Two remedies exist. The generator can send requests on an open-loop schedule and time each one from its intended send time, so delays caused by the server's backlog are charged to the server. Alternatively, a closed-loop tool's histogram can be corrected for the expected interval, adding the samples that the schedule called for during a long response; HdrHistogram supports this correction (Tene, 2012). Exercise 1 Part A reproduces coordinated omission in a simulator and compares the corrected and uncorrected percentiles.

14.3.4 The Latency and Throughput Curve

A single load level gives a single point. Sweeping the offered load from light to heavy and plotting a high percentile against achieved throughput produces a curve that is nearly flat at first and then bends sharply upward as the service approaches its capacity. The bend is called the knee. Below it, extra traffic costs little latency; beyond it, small increases in traffic produce large increases in waiting time, and achieved throughput stops tracking offered load. A latency target is only meaningful at a stated point on this curve, and the margin between expected peak load and the knee is the service's real headroom. Section 14.5 explains the shape of the curve.

One more trap is worth naming. Gregg (2012) calls it the streetlight anti-method: analyzing performance with whichever tools happen to be familiar or at hand. A team that benchmarks only the model because a model profiler is installed will optimize the model whether or not it is the problem. Section 14.4 replaces that habit with systematic attribution.

14.4 Where the Time Goes

Once the service can be measured honestly, the next question is which part of it makes the slow requests slow. A ranking request contains far more operations than its architecture diagram suggests. Chapter 4 cited Etsy's description of search ranking, which scores on the order of 1,000 candidates with about 300 features each inside a budget of a few hundred milliseconds (Walecka and Gallatin, 2023), hundreds of thousands of feature values per request. Guessing which operations dominate the tail is unreliable, so this section introduces an attribution framework and the tools to apply it.

14.4.1 The Critical Path

A recommendation request runs some stages one after another and others side by side. The Chapter 13 orchestrator fetches user features and calls retrieval concurrently (Section 13.8.2), then fetches item features in one batched lookup, runs the light and heavy rankers, and re-ranks, each step waiting for the previous one. Larger systems also query several retrieval sources side by side. The critical path is the chain of work whose duration determines when the response is ready: the serial stages plus, at each parallel step, whichever branch finished last.

Two consequences follow. A set of parallel branches finishes at the slowest one, so its tail is worse than the tail of any single branch: if three independent sources each exceed their p99 one time in a hundred, the step as a whole is slow about three times in a hundred. And work on a branch that rarely finishes last is slack. Making that branch faster changes nothing end to end, however impressive the stage-level improvement looks.

14.4.2 Traces and Spans

The tool that reveals the critical path is distributed tracing. A trace records one request's journey as a tree of spans, where each span is a named, timed unit of work with a start time, an end time, its own identifier, the trace's identifier, and the identifier of its parent span. Sigelman et al. (2010) described this model in Dapper, Google's tracing system, and showed that recording traces for only a sample of requests keeps the overhead small enough to leave tracing on in production. OpenTelemetry, an open standard and set of libraries for telemetry, implements the same model, with context propagation carrying the trace identifier across process boundaries so that spans from different services assemble into one trace (OpenTelemetry, 2026).

Figure 14.3 draws one trace from a system with three retrieval sources as a waterfall. The two-tower search finishes last, so it lies on the critical path, and every serial stage after the merge is on it by construction.

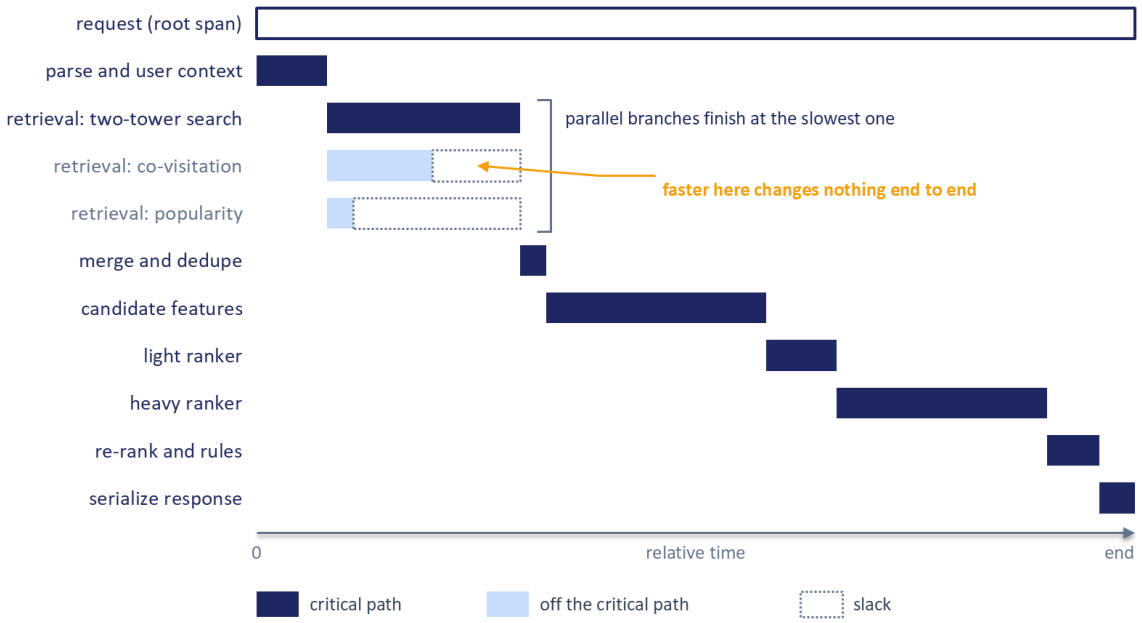


Figure 14.3: An illustrative trace of one recommendation request drawn as a span waterfall. Dark bars lie on the critical path; the co-visitation and popularity retrieval branches finish before the two-tower search, and their dotted slack shows that speeding them up would not change the response time. Proportions are illustrative.

14.4.3 Conditioning on Slow Requests

A common mistake is to read each stage's own p99 and optimize the stage with the largest one. That ranking answers the wrong question. A stage's p99 describes its own slowest requests, which may not be the requests that are slow end to end, and the stage percentiles do not add up to the request's percentile (Section 14.2.3). The useful question is conditional: among requests whose end-to-end latency exceeds the target percentile, how is their critical-path time divided among stages?

Answering it means selecting traces of slow requests and averaging their critical-path composition. The answer often differs from the composition of typical requests. Figure 14.4 contrasts the two views: near the median, time is spread across retrieval, features, and the heavy ranker, while above the p99 a feature-fetch tail dominates. Optimizing the heavy ranker on the strength of the median view would leave the slow requests as they were.

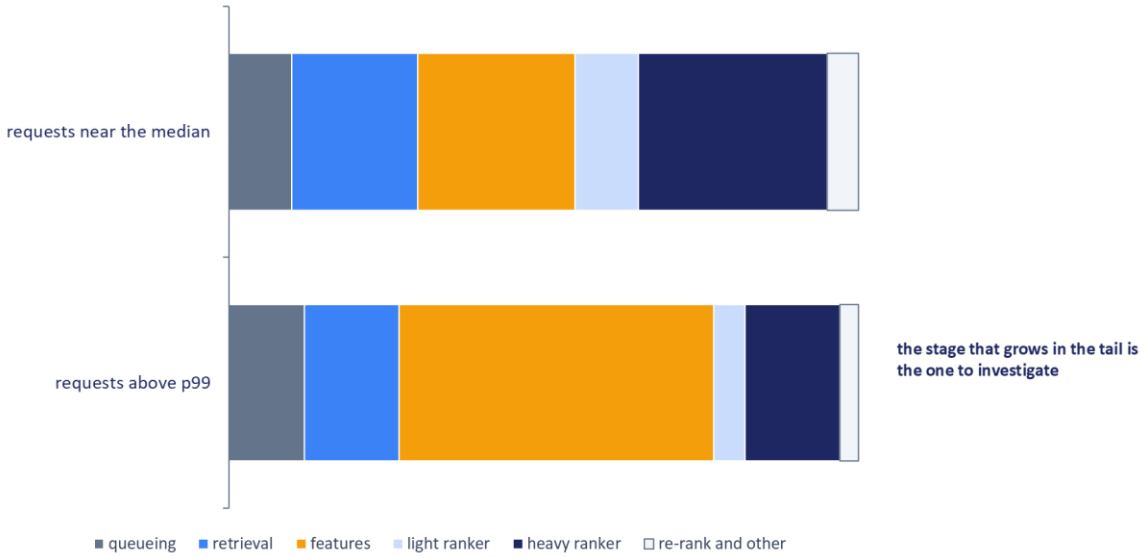


Figure 14.4: Illustrative critical-path composition of requests near the median and requests above the p99. The stage whose share grows in the tail, here feature retrieval, is the one to investigate first. Exercise 1 Part B computes this view from the companion pipeline's traces.

14.4.4 An Attribution Framework

Table 14.2 condenses the framework this chapter uses to attribute time. It partitions a request's critical path into stages S0 through S7, from client transit to response assembly. Within each stage, time splits into service time and waiting time, as defined in Section 14.2.1. A final row, R, holds the unattributed residual: end-to-end time minus the sum of critical-path spans. The residual keeps the partition exhaustive and also measures the instrumentation, since a large residual means the traces have gaps, often in middleware or before the handler starts. Causes that cut across stages, such as garbage collection pauses, cold starts, and contention for cores, are attributed to the stage in which they appear.

Table 14.2: A latency attribution framework for multi-stage recommenders.

Stage	Main cost drivers	First measurement	First levers	Quality risk
S0 Client and network	Distance, mobile networks, connection setup, payload size	Client timing minus server response time	Connection reuse, payload trimming, prefetch	Prefetched results may be stale
S1 Admission and queueing	Utilization, burstiness, concurrency limits, blocking work on an event loop	Timestamps at accept and at handler start; queue depth	Capacity, per-instance concurrency, moving CPU work off the loop, shedding	Shed requests get fallbacks
S2 Orchestrati	Serial remote calls, repeated lookups	Count of serial calls on the critical path	Parallelize independent calls, cache user context	Cached context may be stale

Stage	Main cost drivers	First measurement	First levers	Quality risk
on and context				
S3 Candidate retrieval	Index parameters, filters, number of sources, slowest shard	Span per source and shard; recall against exact search	Search parameters, candidate count, hedging, precomputed candidates	Lower recall, weaker long-tail coverage
S4 Feature retrieval	Candidates times features, round trips, store tail, cache misses	Span per fetch; keys and bytes per request; hit rate	Batched reads, feature cache, precomputation, trimming unused features	Staleness and training-serving skew
S5 Model inference	Compute, embedding lookups, batch size, threads, hardware	Span per model; batch-size distribution; operator profile	Batching, graph optimization, quantization, distillation, fewer candidates	Score drift and rank changes
S6 Re-ranking and rules	Algorithm cost in candidates and slate size, remote checks	Span; scaling with candidate count	Vectorized code, fewer candidates, earlier eligibility checks	Less diversity headroom
S7 Response and logging	Payload size, text encoding, synchronous logging	Span; bytes per response	Binary encoding, asynchronous logging	Lost logs weaken training data
R Residual	Untraced middleware, clock differences	End-to-end time minus span sum	Add spans until small	None

Sections 14.6 to 14.9 develop the levers, Section 14.10 explains how to choose among them, and Exercise 1 Part B (Section 14.13) tests six of them one at a time.

14.4.5 Checking the Resources

Traces show which stage is slow; they do not always show why. Gregg (2012) proposes the USE method as a complement: for every resource, such as each CPU, memory, disk, network interface, and software pool such as a thread pool or connection pool, check utilization (the fraction of time the resource is busy), saturation (work waiting for the resource, often a queue length), and errors. A feature-fetch span that is long because a connection pool is exhausted shows up as saturation on that pool, and adding a cache would treat the wrong cause. Gregg also warns that utilization averaged over long intervals hides

short bursts at full utilization, which is exactly when tail requests occur, so resource metrics for tail analysis need short intervals.

14.5 Why Tails Grow: Queueing, Fan-Out, and Variability

Two facts surprise teams the first time they load-test a recommender: a service that is fast at moderate load becomes slow well before its processors are fully busy, and a request that calls many backends is slow far more often than any one backend. Both follow from short arguments.

14.5.1 Utilization and Queueing

Three quantities describe load on a server. The arrival rate, written λ, is the number of requests arriving per second. The service rate, μ, is the number the server can complete per second when it is never idle, so $1/\mu$ is the mean service time. Utilization, $\rho = \lambda/\mu$, is the fraction of time the server is busy.

Little's law relates them to concurrency. If L is the average number of requests in the system and W is the average time each spends there, then

$$L = \lambda W$$

for any stable system, whatever the distributions involved (Little, 1961). A service handling 200 requests per second with a mean response time of 50 ms holds 10 requests in flight on average. The law sizes connection pools and worker counts, and it shows that longer response times at fixed traffic mean more requests in flight competing for the same resources.

The simplest model of queueing has one server, requests arriving at random (a Poisson process), and service times drawn from an exponential distribution. For this model, the mean response time is

$$T = \frac{1}{\mu - \lambda} = \frac{1/\mu}{1 - \rho},$$

and the response time itself is exponentially distributed, so its 99th percentile is $\ln(100)$ times the mean (Harchol-Balter, 2013). The factor $1/(1-\rho)$ carries the whole story. At 40% utilization the mean response time is 1.67 service times; at 80% it is 5, three times as long, with no change to the code. Figure 14.5 plots the curve, whose steep rise as utilization approaches one is the knee that Section 14.3.4 described.

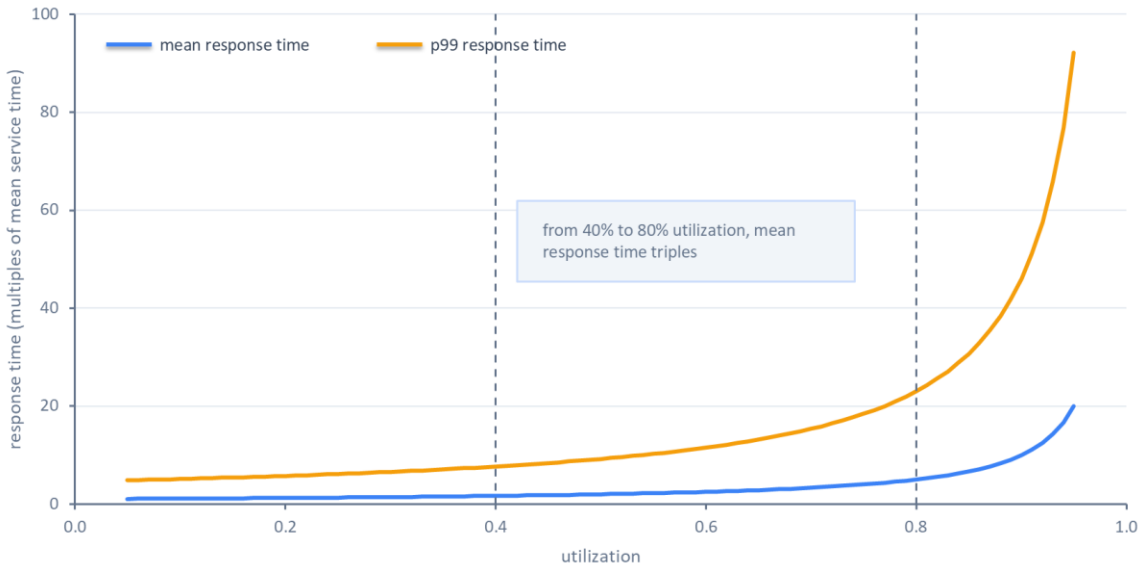


Figure 14.5: Mean and 99th-percentile response time against utilization for a single server with random arrivals and exponential service times, in multiples of the mean service time. Moving from 40% to 80% utilization triples the mean response time in this model (Harchol-Balter, 2013). Computed from the formula, not measured.

Real services differ from the model in ways that make matters worse. Service times vary more than an exponential distribution when some requests carry far more candidates, hit cold caches, or trigger a garbage-collection pause, and Harchol-Balter (2013) shows that waiting time grows with the variability of service times as well as with utilization. Arrivals are burstier than a Poisson process when traffic follows notifications, scheduled jobs, or retries. A team can reduce waiting time by adding capacity, which lowers utilization, or by reducing variability, which flattens the curve at the same utilization.

Serverless container platforms expose one of these controls directly. The limit on concurrent requests each replica accepts (Section 13.9.3) caps how many requests one instance handles at once, and with it the utilization of each instance: a lower cap sends excess traffic to other instances or new ones, trading cost for a shorter queue. A Python service adds a subtler queue. FastAPI runs plain `def` handlers in a thread pool and `async def` handlers on the event loop (FastAPI, 2026), as Section 13.8.2 noted, so CPU-bound work inside an `async def` handler, such as scoring candidates or running MMR, blocks the loop and every request waiting on it. The time shows up as waiting time in stage S1, far from the stage that caused it.

14.5.2 Fan-Out Amplification

Recommenders fan out. Retrieval queries several sources, embedding lookups may hit many shards of an embedding lookup service (Section 13.5.3 and Figure 13.5), and feature retrieval may span several stores. When a request waits for N parallel calls and each is independently slow with probability q, the probability that the request waits on at least one slow call is

$$1 - (1 - q)^N.$$

Dean and Barroso (2013) gave the canonical example: if each of 100 servers is slow on one request in 100, then 63% of requests that touch all of them are slow. Figure 14.6 plots the expression for three values of q. With enough fan-out, the tail of the components becomes the median of the whole, so a component's p99, or even its p99.9, is what large fan-out requests experience routinely.

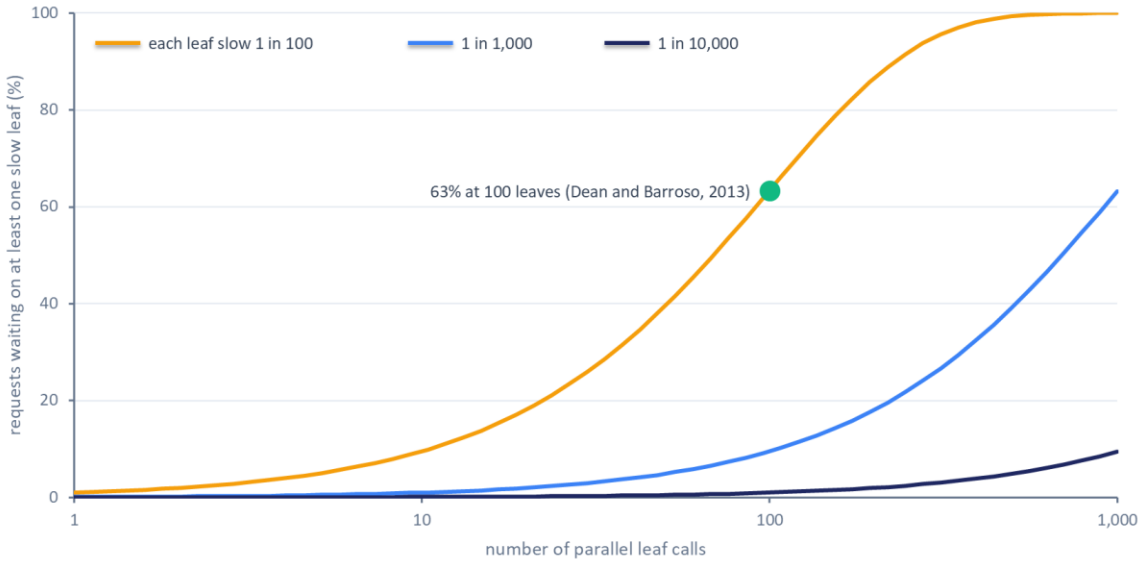


Figure 14.6: The share of requests delayed by at least one slow leaf as fan-out grows, for leaves that are each slow on 1 request in 100, 1 in 1,000, and 1 in 10,000. At 100 leaves slow 1 time in 100, 63% of requests wait on a slow leaf (Dean and Barroso, 2013). The curves are exact for independent leaves. Computed from the formula, not measured.

Variability has many sources besides load. Dean and Barroso (2013) list shared resources contended by co-located work, background daemons, maintenance activity, queueing at several layers, and pauses for garbage collection and background compaction in storage systems. Cold starts and cache misses add more, and since none can be removed entirely, large systems learn to tolerate them.

14.5.3 Tail-Tolerant Techniques

The most widely used technique is the hedged request: send a request to one replica, and if no response has arrived after a delay, send a second copy to another replica and use whichever answers first. Dean and Barroso (2013) recommend waiting until the 95th-percentile expected latency before hedging, which limits the extra load to about 5% of requests while removing much of the tail. In their reported benchmark, reading 1,000 keys spread across 100 servers and hedging after 10 ms cut the 99.9th-percentile latency from 1,800 ms to 74 ms while sending 2% more requests. Hedging needs idempotent requests and spare capacity; it helps when slowness is a property of the replica at that moment and does little when every replica is overloaded.

Dean and Barroso (2013) describe several related techniques. Tied requests enqueue copies on two servers that cancel each other once one starts work. Micro-partitions split data into many more pieces than machines so that load can be rebalanced finely, and selective replication adds copies of hot partitions. Latency-induced probation removes slow machines from rotation until they recover. Good-enough results return an answer once a sufficient fraction of leaves has responded, which suits retrieval, where missing one source among several costs a little recall. Exercise 1 Part A reproduces Figures 14.5 and 14.6 in the simulator and measures, at several utilizations, the extra requests hedging sends and what it does to the tail.

14.6 Doing Less Work per Request

The cheapest millisecond is the one the request never spends. Before making any stage faster, it is worth asking which work could leave the request path altogether: computed earlier, reused from a previous request, done for fewer candidates, or stopped once the answer is good enough. Each of these levers removes latency at a price, and the price is usually freshness or recall.

14.6.1 Precomputation

Amatriain and Basilico (2013) described Netflix's recommendation architecture in terms of where computation happens. Offline computation runs in batch with relaxed time limits, so it can use complex algorithms and large amounts of data, but its results grow stale between runs. Online computation responds to the current request and the most recent events, at the cost of a strict response-time limit. Nearline computation sits between them: it runs in response to events, like online computation, and stores its results for later requests instead of returning them immediately. The same authors stressed that online computation needs a fast fallback, such as a precomputed result, for the occasions when it cannot finish in time.

Precomputation moves work from the online column to the other two. Netflix's personalization systems periodically compute recommendations for every member and store them in a cache tier for fast retrieval (Netflix Technology Blog, 2018). LinkedIn built batch-computed recommendation data into read-only stores offline and swapped each new version into the serving system as a whole, so bulk loading never competed with live requests and a bad version could be rolled back quickly (Sumbaly et al., 2012). Chapter 4 made the same choice at the level of individual features, deciding per feature whether to compute it on the write path or at request time (Section 4.2 and Figure 4.2). The cost is the one Section 4.8 called the freshness dividend run in reverse: a precomputed list cannot react to what the user did a minute ago. Many systems therefore precompute the expensive, slowly changing parts, such as user embeddings, candidate pools, and item features, and leave a light online step to react to the session.

14.6.2 Caching

A cache stores the result of earlier work so a later request can reuse it. A recommender can cache at several levels. A response cache stores the final slate for a key. A user-embedding cache stores the output of the user tower. A feature cache stores fetched feature values, and a prediction cache stores model outputs so that a repeated query is not scored twice; Crankshaw et al. (2017) built such a cache into Clipper, their model-serving system, in front of the models it serves.

The level matters because personalized responses cache poorly. If the key includes the user, every user needs their own entry and a request hits the cache only when the same user returns before the entry expires. User embeddings and item features are the usual compromise: they are reused across many candidates or many requests, and each fetch saves a round trip. Whatever the level, the key must include everything that changes the answer. A response cache keyed without the user, the experiment arm, or the eligibility context can serve one user's recommendations to another or leak a treatment into the control group.

How large a cache must be depends on how skewed the popularity of keys is, and that is a measurement. Breslau et al. (1999) found that web requests across six proxy traces followed Zipf-like distributions, in which the request share of the i-th most popular item is proportional to $1/i^{\alpha}$, with an

exponent α that varied from trace to trace, and that hit ratio grew roughly logarithmically with cache size. Figure 14.7 shows how strongly the exponent sets the hit rate at a given cache size.

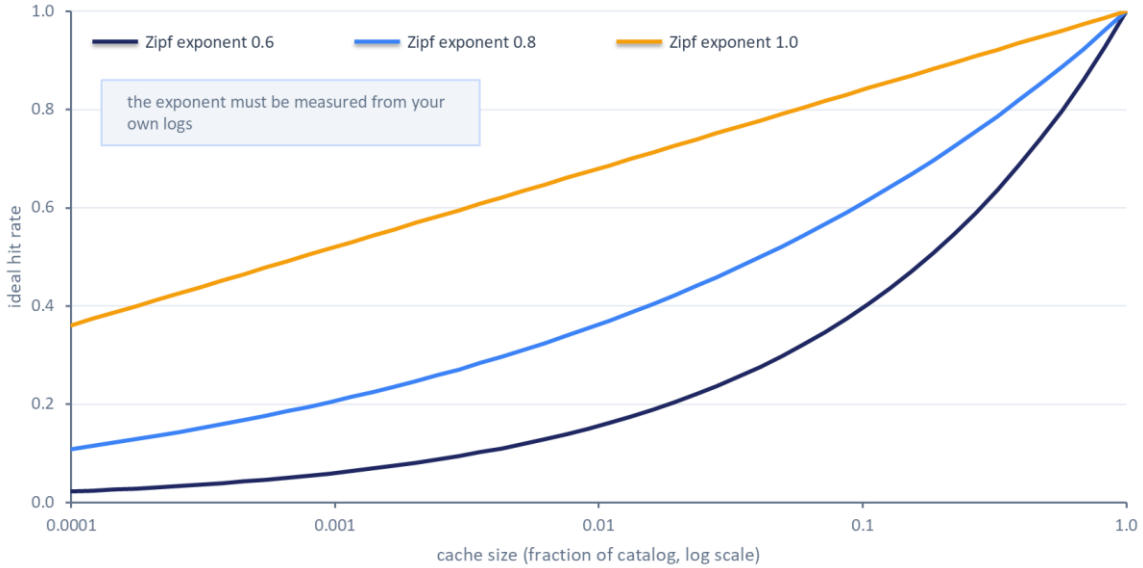


Figure 14.7: Ideal cache hit rate against cache size for a catalog of one million items whose popularity follows Zipf-like distributions with three exponents. The curves assume static popularity, independent requests, and a cache that always holds the most popular items, so real hit rates are lower; Breslau et al. (1999) found the exponent varies across request traces. Computed from the formula, not measured.

Caches also bring their own tail. A cache miss is slower than having no cache, because it pays for the lookup and then for the work, and misses cluster on new users, new items, and the long tail, which are already the slices where recommendation quality is weakest. When an entry expires or is invalidated, many concurrent requests may miss at once and stampede the backend. Section 14.11 returns to checking that a cache has not changed the recommendations for those slices.

14.6.3 Candidate Budgets and Early Termination

The number of candidates that flows from retrieval to ranking multiplies the cost of every later stage: feature retrieval scales with candidates times features, ranking with candidates times model cost, and re-ranking algorithms such as MMR with candidates times slate size (Section 13.6.3). The candidate budget, the number of candidates passed from each stage to the next, is therefore one of the most effective latency levers, and also a direct quality lever, since a candidate that is never retrieved can never be ranked.

Large systems manage the budget in stages. Instagram's Explore system samples 500 candidates per request from thousands of eligible items, then a distilled first-pass model with minimal features picks 150, a lightweight neural network with a full set of dense features picks 50, and a deep model with dense and sparse features picks the final 25 (Medvedev, 2019). Each stage spends more per candidate on fewer candidates. The right budget for each stage is found by sweeping it and measuring both latency and downstream quality, as Section 14.7 does for retrieval; Exercise 1 Part B's `candidates` lever measures the latency side.

Early termination stops work once the answer is good enough. Pinterest's Pixie recommends items by running random walks on a graph of pins and boards and counting visits. Instead of running a fixed number of steps, it stops once enough candidates have reached a minimum visit count, which produced

almost the same results as long walks in about half the steps (Eksombatchai et al., 2018). The general pattern is to replace a fixed amount of work with a stopping rule tied to the quality of the answer so far, and to verify the rule against the full computation before trusting it.

14.7 Faster Retrieval: Choosing an Operating Point

Chapter 5 explained how approximate nearest-neighbor indexes work: an IVF index partitions the vectors into clusters and searches only the closest few, an HNSW index walks a layered proximity graph, and product quantization compresses vectors into short codes (Section 5.5 and Figures 5.5 and 5.6). That chapter stated the recall and latency trade-off as a principle. This section turns it into a measurement and a decision.

14.7.1 Search-Time Parameters and Recall

Every approximate index exposes search-time parameters that trade speed for accuracy without rebuilding the index. For IVF, `nprobe` sets how many clusters a query searches; for HNSW, `efSearch` sets how many candidate nodes the graph walk keeps in its working list. Raising either makes search slower and more accurate. Douze et al. (2024), describing the Faiss library, frame the choice as one among settings of these parameters and treat search time, memory use, and index build time as the resources being traded against accuracy.

Accuracy here means recall@K against exact search: for a set of evaluation queries, the fraction of the true K nearest neighbors, found by brute-force search, that the approximate index also returns. Aumüller et al. (2020), whose ANN-Benchmarks suite is a widely used public comparison, evaluate indexes by plotting recall against queries per second across many settings, so that each index appears as a curve.

Recall against exact search is a proxy for whether the final recommendations are as good. The downstream ranker may forgive missing neighbors that it would have ranked low anyway, or it may not, if the missing items are the long-tail candidates that make recommendations feel personal. A retrieval evaluation therefore pairs recall@K with an end-to-end metric, such as the hit rate of the user's next interaction among the final recommendations, measured at the same settings.

14.7.2 The Pareto Frontier

Measuring each candidate configuration at several parameter values produces a cloud of points in the plane of latency and recall. A setting is dominated if another setting is at least as fast and at least as accurate, and better on one of the two. The settings that no other setting dominates form the Pareto frontier, and Douze et al. (2024) note that only Pareto-optimal settings are worth keeping: those that are fastest for a given accuracy, or equivalently most accurate for a given time budget. When several parameters interact, the number of combinations grows quickly, and Faiss's tuning tools exploit the fact that raising a parameter never makes search faster to prune settings that cannot reach the frontier.

The operating point is the setting the service actually runs. Choosing it takes two lines drawn from outside retrieval. The first is a recall floor, set from downstream quality: the lowest recall at which the end-to-end metric stays within an acceptable distance of exact search. The second is the retrieval stage's latency allowance at the target percentile, which Section 14.9 derives from the overall latency target. The chosen point is the fastest frontier setting above the floor, provided it falls inside the allowance. Figure 14.8 shows the construction with two index configurations whose frontiers cross and a third that

never reaches the frontier. Neither of the first two dominates the other everywhere, so the choice depends on where the floor sits.

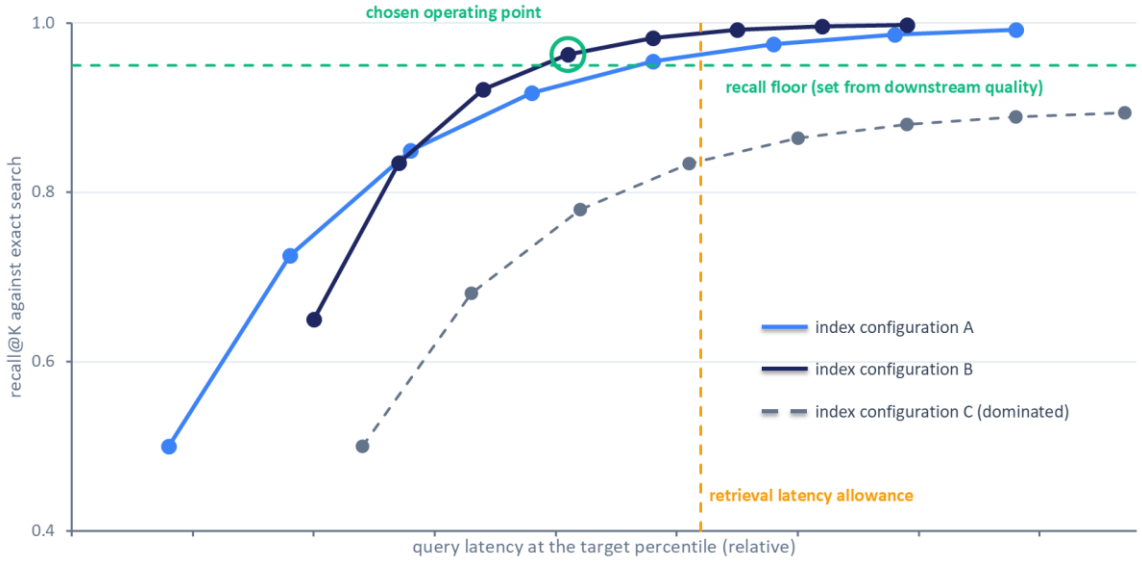


Figure 14.8: Choosing an approximate search operating point. Each curve traces one index configuration across values of its search-time parameter. Configurations A and B cross, so neither dominates the other; configuration C is dominated at every setting. The chosen point is the lowest-latency setting that meets the recall floor, which is set from downstream quality, and falls inside the retrieval latency allowance. The curves are illustrative; real frontiers depend on the data, the hardware, and the load.

Three details change the frontier. Latency should be measured at the target percentile and the expected load, since a setting that looks fast in a single-query loop may queue under concurrent traffic. Query batching and thread counts move the whole curve, often in opposite directions for throughput and single-query latency. And retrieving 1,000 candidates costs more than retrieving 100, so the frontier must be measured at the candidate count K the ranker actually consumes (Section 14.6.3).

14.7.3 Compression and Adaptive Retrieval

Product quantization and related compression schemes shrink each vector into a short code, so more of the index fits in memory and each distance computation reads less data. The price is lower recall at the same parameter settings, and changing the compression requires rebuilding the index, which makes it a slower lever to adjust than a search-time parameter. A common pattern retrieves a larger candidate set with compressed codes and re-scores it with full-precision vectors.

Matryoshka representation learning applies the same pattern inside the embedding itself. Kusupati et al. (2022) train embeddings so that their leading dimensions form a usable lower-dimensional embedding. Adaptive retrieval then shortlists candidates using a short prefix of each vector and re-ranks the shortlist with the full vector. On image retrieval benchmarks, the authors reported large wall-clock speedups at comparable accuracy. The approach requires training with the nested objective, and whether a recommender's embeddings tolerate a short prefix has to be measured on its own data.

14.7.4 Filtered Search

Recommenders rarely search the whole catalog: a request may need items in stock, in a region, in a category, or not already seen. Combining a vector search with a predicate like this is called filtered or

hybrid search, and the two simple strategies have opposite failure modes. Pre-filtering first finds every item that satisfies the predicate and then searches only those, which guarantees correct results but costs time that grows linearly with the number of matching items. Post-filtering searches the full index and discards results that fail the predicate, which is fast when matching items are common and close to the query, and expensive when they are rare or far from it, because the search must widen until enough matches survive. Patel et al. (2024) show that the cost of post-filtering depends on the predicate's selectivity, the fraction of items that pass it, and on the correlation between the predicate and the query, and they propose ACORN, a graph index whose search traverses only the subgraph of items that pass the predicate. Filtered queries therefore form their own slice, with a tail that can differ greatly from unfiltered queries, and they belong in the measurement set.

Multi-source retrieval adds one more constraint. When several sources run in parallel, the step finishes at the slowest source (Section 14.4.1), so tuning the fastest source's operating point buys nothing; the slow source's tail is the target, and hedging it (Section 14.5.3) or letting the merge proceed without it (Section 14.9) are the levers. Exercise 2 Part A (Section 14.13) draws Pareto frontiers for four index types at three values of K.

14.8 Faster Ranking: Batching, Compilation, Compression, and Hardware

Chapter 13 deferred batching, quantization, distillation, and infrastructure tuning to this chapter (Section 13.12). All act on stage S5, model inference. Some change only how the same computation is scheduled; others change the model's outputs and need the verification of Section 14.11.

14.8.1 Memory-Bound and Compute-Bound Work

A computation is compute-bound when its speed is limited by arithmetic throughput and memory-bound when it is limited by how fast data reaches the processor. Recommendation models contain both. Their dense layers, the multilayer perceptrons and cross networks of Chapters 6 and 7, are mostly compute-bound. Their embedding lookups are memory-bound: each lookup reads a few rows scattered across very large tables and does little arithmetic on them. Gupta et al. (2020a) found that recommendation models accounted for up to 79% of the AI inference cycles in Facebook's data centers, that embedding operations had low compute density and high cache-miss rates, and that the share of time in fully connected layers varied widely across models. Lui et al. (2021) reported embedding tables as over 97% of model size for the models they studied. Faster arithmetic therefore does little for a model dominated by embedding lookups, while smaller embeddings or fewer lookups help a great deal.

14.8.2 Batching

Every call into a model pays fixed costs before it computes anything: crossing from Python into native code, dispatching operators, allocating outputs, and on an accelerator, launching kernels and copying inputs. Intra-request batching scores all of a request's candidates in one call so the fixed cost is paid once. Chapter 13's light ranker does this with a single batched XGBoost predict call over the candidate set (Section 13.4.3), and scoring candidates one at a time in a loop is a common source of avoidable latency.

Inter-request batching, usually called dynamic batching, goes further: the server holds requests for a short window and merges their candidates into a larger batch (Section 13.5.4). Larger batches raise throughput, but each request now waits for the batch to form. Olston et al. (2017) note in their description of TensorFlow Serving that inter-request batching has to be managed carefully to avoid

unduly hurting latency. Crankshaw et al. (2017) made the batch size adaptive in Clipper, increasing it additively until a batch's latency exceeds the latency objective and then reducing it by a fixed fraction, and found that the largest batch fitting a given objective differed by orders of magnitude between models.

Figure 14.9 shows the trade-off under a simple cost model in which time per call is a fixed overhead plus a per-item cost. When fixed overhead dominates, throughput keeps improving with batch size up to a large batch; when per-item cost dominates, the latency allowance caps the batch early. The quantity to maximize is latency-bounded throughput, the most items per second achievable while each call stays inside its allowance, which Gupta et al. (2020b) adopted as the objective of DeepRecSys, their scheduler for recommendation inference.



Figure 14.9: Time per call (left) and throughput (right) against batch size for two illustrative cost models, one dominated by fixed per-call overhead and one by per-item cost. A per-call latency allowance caps the batch size, and the markers show the latency-bounded throughput each model reaches. Waiting to fill a batch adds latency when traffic is light. Real curves depend on the model and hardware (Gupta et al., 2020b; Eksombatchai and Zhang, 2022).

The best batch size is rarely at either extreme. DoorDash's prediction service split each request's stores into chunks, and its load tests found a middle range of chunk sizes fastest: very small chunks were slower, which the team attributed to the extra requests queueing, while a single large request was slowed, they suggested, by its payload size (Zeng, 2020). Where the middle lies has to be found by sweeping.

14.8.3 Export and Graph Optimization

A model trained in a framework such as PyTorch can be exported to a portable format such as ONNX and run by an inference runtime that optimizes the computation graph before executing it. ONNX Runtime applies optimizations at levels: basic optimizations such as constant folding, removal of redundant nodes, and fusions that preserve semantics; extended optimizations that fuse larger patterns, such as a matrix multiplication followed by an addition or an activation, on supported hardware; and layout optimizations that rearrange data for particular processors (ONNX Runtime, 2026a). Fusion reduces the per-operator overhead on which small recommendation models spend much of their time.

The runtime can optimize a model when it loads or save the optimized model offline, and an offline-optimized model is tied to the options and hardware it was optimized for. Export itself can introduce silent differences, such as a dropped preprocessing step, so even this nominally output-preserving lever needs a score parity check.

14.8.4 Compression

Quantization represents weights, activations, or both with low-precision integers instead of 32-bit floating point. The standard affine scheme maps each real value to an integer through a scale s and a zero point z (Jacob et al., 2018):

$$x_{\text{real}} = s(x_{\text{quantized}} - z).$$

Static quantization fixes the activation scales in advance from a calibration set; dynamic quantization computes them for each input as the model runs, which is usually more accurate and adds overhead (ONNX Runtime, 2026b). The speedup is not guaranteed. ONNX Runtime's documentation states that the gain depends on the model and the hardware and that slower performance on older processors is not rare; processors with dedicated integer dot-product instructions benefit most (ONNX Runtime, 2026b). For embedding-dominated models, quantizing the tables addresses the memory-bound part directly: Guan et al. (2019) reduced a production model to under 14% of its original size with post-training 4-bit row-wise quantization of its embedding tables, at neutral quality in their evaluation.

Distillation trains a smaller student model to reproduce a larger teacher's outputs (Hinton et al., 2015). Tang and Wang (2018) adapted it to ranking, and on their public datasets students under half the teacher's size reached similar ranking quality. Instagram's Explore system trained its lightweight first-pass ranker on recorded inputs and outputs of the heavier later stages, so that it approximates their ordering with far fewer features (Medvedev, 2019). Chapter 5 (Section 5.8.3) applied distillation to retrieval, and Chapter 7 (Section 7.10.1) mentioned it as a latency tool.

Structured pruning removes whole components instead of individual weights, so the remaining computation stays dense and fast. Deng et al. (2021) pruned the network, field-interaction, and embedding components of a click-through-rate model and reported speedups of tens of times on the Criteo and Avazu benchmarks without loss of AUC. Compositional embeddings reduce embedding memory by building each category's vector from several smaller tables, using complementary partitions such as the quotient and remainder of the category index; Shi et al. (2020) found this preserved model quality better than hashing categories into a smaller table. Both require retraining, and both change outputs.

14.8.5 Hardware and Threads

Moving inference to an accelerator such as a GPU pays when batches are large enough to amortize transfer and launch costs. Gupta et al. (2020b) found that the batch size at which an accelerator became faster than a CPU depended on the model, the load, and the tail-latency target, and DeepRecSys routed queries above a size threshold to the accelerator. Pinterest's move to GPU serving shows the practical side (Eksombatchai and Zhang, 2022). At small serving batch sizes, launching many small kernels cost more than the computation, so the team merged operations, consolidated hundreds of host-to-device copies into one, and captured inference as a static CUDA graph, which required padding inputs to fixed shapes. It also inverted its batching strategy: on CPUs it had split requests into small batches across machines for latency, while on GPUs it formed larger batches. Pinterest's later account describes batching work across user requests and half-precision inference as part of the efficiency work that made GPU serving viable (Pinterest Engineering, 2025). On CPUs, thread settings deserve the same scrutiny as

hardware: the number of threads per inference call and the number of concurrent calls trade single-request latency against throughput, and oversubscribed threads (Section 14.3.1) degrade both. Exercise 2 Part B (Section 14.13) measures batch sizes and thread counts on a CPU.

14.9 Budgets, Deadlines, and Graceful Degradation

Optimization makes the typical request faster; it does not guarantee that every request finishes in time. The remaining work is to divide the latency target among stages, make every stage respect its share, and decide in advance what the service returns when a stage cannot.

14.9.1 Allowances from Measurements

A latency allowance is the share of the end-to-end latency target assigned to one stage. Chapter 13 divided an example 200 ms target among its stages as a planning tool and noted that stage budgets do not add (Section 13.1.3 and Figure 13.2). That allocation is a reasonable starting point for a system that has not yet been measured. Once traces exist, the allocation should come from them: the allowance for each stage on the critical path starts from its measured distribution in requests near the target percentile, and the slack between the sum of those shares and the target is assigned to the stages where extra time buys the most quality, usually ranking.

Trace replay supports this reasoning. Replace one stage's spans in recorded traces with samples from a faster distribution, such as one a proposed optimization achieved in a microbenchmark, recompute each request's critical path, and read off the new end-to-end percentile. The replay answers "what if this stage were faster?" without building the change and respects the parallel structure. It ignores load effects, since a faster stage also frees capacity and shortens queues elsewhere, so its estimate is a guide to confirm under load.

Stage times quoted in Chapter 2 and Chapters 5 through 8 are illustrative and describe different systems, models, and candidate counts. No published stage latency substitutes for allowances derived from a team's own measurements, and this chapter repeats none of them.

14.9.2 Timeout Values and Retry Budgets

Chapter 13 built the mechanism: a deadline set once per request, the remaining time passed to every call, and per-call timeouts derived from it (Section 13.8.3). This section chooses the numbers.

Timeout values should come from the downstream distribution. Brooker (2019) recommends choosing an acceptable rate of false timeouts, meaning calls abandoned that would have succeeded, and setting the timeout at the corresponding percentile of the downstream latency: a 0.1% false-timeout rate means the p99.9. Tight distributions need padding, because their percentiles sit close together. Brooker also describes timeouts that fired after deployments because connection setup counted against a short timeout, fixed by establishing connections before taking traffic, a cold-start effect of the kind Section 14.3.1 described.

Chapter 13 also set the retry rules: a single retry point, capped exponential backoff with random jitter, and a process-wide retry budget (Section 13.8.4). Hedged requests (Section 14.5.3) are a form of retry and draw on the same budget. Hedging after the downstream p95 adds at most about 5% more calls (Dean and Barroso, 2013), and fewer when hedging also shortens the queues behind the slow calls, so the budget must cover that share at peak load on top of genuine retries, or hedging will be cut off exactly when the tail is worst.

14.9.3 Budget- and Load-Triggered Degradation

Chapter 13's degradation ladder (Section 13.8.5 and Figure 13.6) reacts to failures: when a stage fails or its circuit breaker opens, the orchestrator falls back to a defined cheaper path. A request can also run short of time with every stage healthy, or arrive while the service is near capacity. This section extends the ladder with levels triggered by the remaining budget or by load, each removing a known piece of work at a quality cost measured beforehand. Figure 14.10 shows how a propagated deadline drives the first such level: before the heavy ranker, the orchestrator checks whether the remaining budget covers the ranker's allowance.

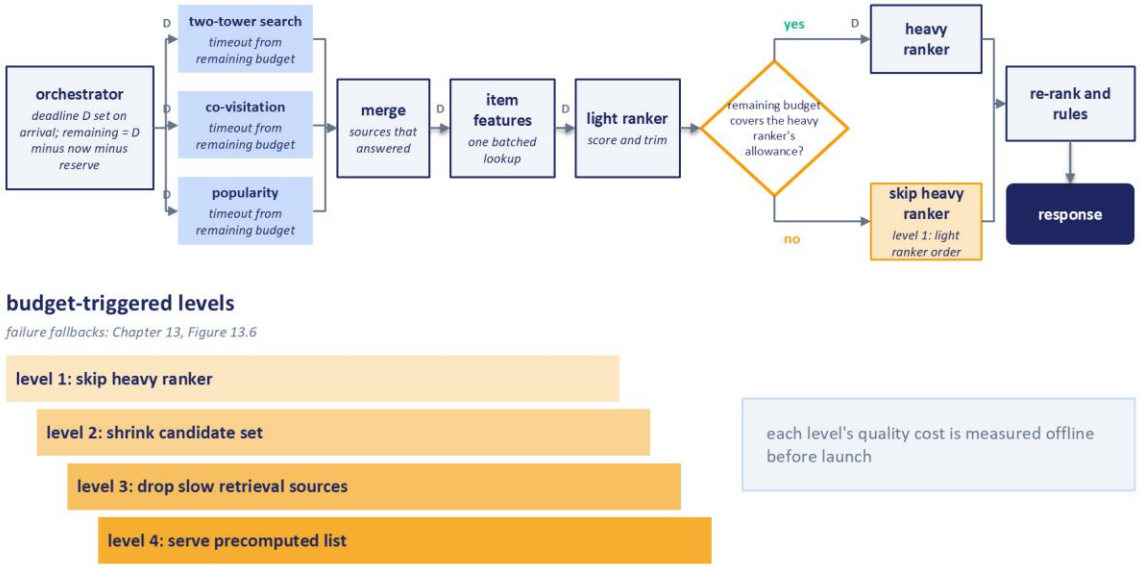


Figure 14.10: A propagated deadline driving per-stage timeouts and budget-triggered degradation. The orchestrator passes the deadline to every call and computes the remaining budget before each stage. The merge uses the retrieval sources that answered in time, and when too little budget remains for the heavy ranker, the response keeps the light ranker's order. The levels below are cheaper responses whose quality costs are measured offline before launch; Chapter 13's failure fallbacks (Figure 13.6) handle stages that fail.

Table 14.3 lists four such levels. Their triggers are conditions on the remaining budget, on load, or on timeouts, never fixed numbers, because the right thresholds come from the allowances above.

Table 14.3: Budget- and load-triggered degradation levels.

Level	Action	Work removed	How quality cost is measured beforehand	Trigger
1	Skip the heavy ranker; serve the light ranker's order	Heavy ranker inference	Offline metrics on a replay set, light order against full order	Remaining budget below the heavy ranker's allowance
2	Shrink the candidate set	Part of feature retrieval and ranking	Sweep of candidate counts against offline metrics	Sustained load near capacity

Level	Action	Work removed	How quality cost is measured beforehand	Trigger
3	Drop slow retrieval sources	The slowest source's tail	Recall and offline metrics without each source	Source exceeds its timeout
4	Serve a precomputed list	All online stages	Offline metrics of the precomputed list against the online pipeline	Remaining budget below the minimum online path

Two practices keep these levels trustworthy. First, each level's quality cost is measured offline before launch, so the team knows what a degraded response costs and can decide how often degradation is acceptable. Second, the degraded paths are exercised regularly. Beyer et al. (2016) observe that the code path a system never uses is often the one that does not work, and recommend exercising degraded modes, for example by running some servers near overload. Netflix goes further, classifying requests by priority and shedding the least critical ones first under load, and validates the policy with fault-injection experiments (Correa et al., 2020). Amatriain and Basilico (2013) give the reason a precomputed result belongs at the bottom of the ladder, and Chapter 15 reuses these paths for stage bypass (Section 15.8.4) and monitors how often each fallback and timeout fires in production.

14.10 Choosing What to Optimize

This section turns the levers of Sections 14.6 to 14.9, more than any team can pull at once, into a method for deciding which to try next, in which each step prevents a specific way of wasting effort.

14.10.1 Estimate, Probe, Classify, Cost

Start from a latency target with all five parts (Section 14.2.4) and a baseline measured as Section 14.3 described: open-loop load at the target rate, warmup discarded, repeated runs, and an interval on the target percentile. Then attribute the tail as Section 14.4 did, from traces of requests above the target percentile.

The first step for each candidate lever is an estimate of its gain. A request can directly lose at most the time a stage spends on its critical path, so the stage's share of critical-path time in slow requests estimates what improving it could remove from the target percentile: a stage that takes a fifth of the tail time can remove about a fifth of it even if made infinitely fast, and a stage on a parallel branch that rarely finishes last has an estimate near zero. For a percentile the estimate is first-order: a faster stage changes which requests are slow and, under load, shortens the queue for the requests behind it, so the percentile can fall by more or by less than the share. Trace replay (Section 14.9.1) refines the estimate on recorded traces, with queueing held as it was. The reasoning follows Amdahl (1967), who argued that accelerating part of a computation speeds up the whole by at most the reciprocal of the remaining part; applying it to shares of tail latency is this chapter's synthesis. The estimate is cheap to compute and eliminates most candidates.

The second step is a probe: the cheapest experiment that estimates the real gain on the team's own model, data, and hardware. A parameter sweep, a microbenchmark of an exported model, a hit rate

estimated from logs, or a trace replay (Section 14.9.1) all qualify; a published speedup describes someone else's system.

The third step classifies quality risk. This chapter uses four classes. R0 levers leave outputs identical up to floating-point rounding: connection reuse, parallelizing independent calls, thread settings, and exact export. R1 levers change outputs in a bounded way that offline measurement can quantify: search parameters, quantization, index compression, and candidate counts checked against offline metrics. R2 levers can be measured offline only partially and need slice checks and shadow comparison: distillation, pruning, and caches that introduce staleness. R3 levers need an online experiment, because their effect depends on how users respond: large candidate cuts or reductions in freshness.

The fourth step estimates cost: engineering effort, running cost in hardware and memory, operational complexity, and reversibility, which ranges from a configuration flag to a retrained model or a rebuilt index. Within the lowest risk class available, levers are ordered by probed gain per unit of cost, and reversible changes win ties. Apply one change, then measure again, because the bottleneck moves: once the heavy ranker is faster, feature retrieval may dominate the tail. A simple worksheet records, for each lever, the stage, its tail share, the probed gain with its interval, the risk class, the cost, and the reversibility. Figure 14.11 draws the same flow.

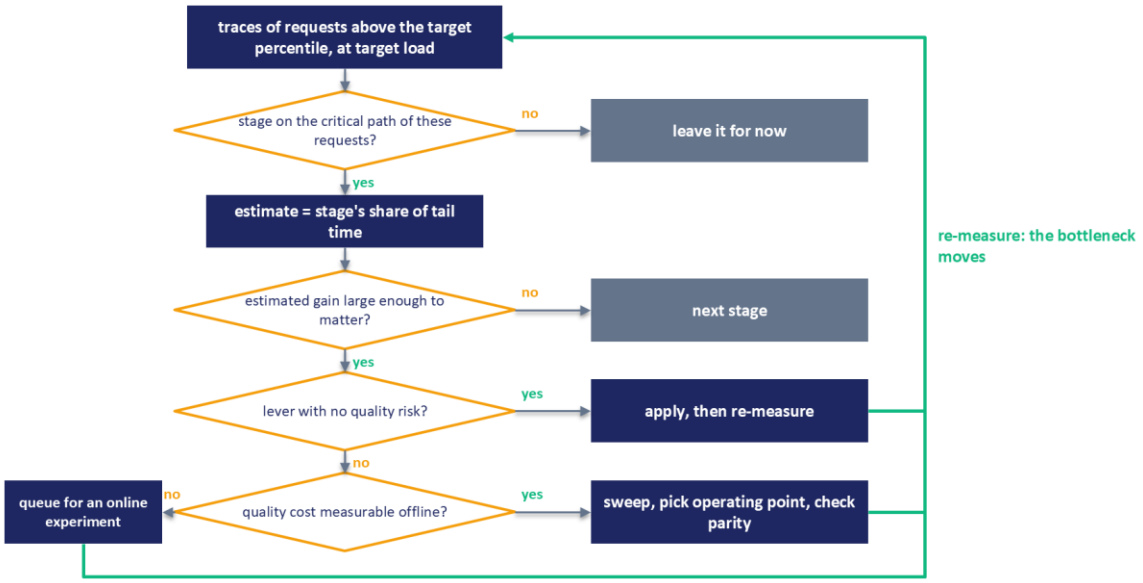


Figure 14.11: A triage flow for choosing the next optimization. Starting from traces of slow requests at target load, each stage is checked for whether it lies on the critical path, whether its share of tail time suggests a worthwhile gain, and what quality risk its lever carries. Each applied change leads back to measurement, because the bottleneck moves; stages that fail a test wait for the next pass.

Table 14.4 summarizes the levers from earlier sections along the dimensions this method uses.

Table 14.4: Optimization levers compared.

Lever	Time removed	Risk class	Verification required	Reversibility	Main cost
Connection reuse, batched reads	Setup and round trips	R0	Latency benchmark	Configuration	Low effort

Lever	Time removed	Risk class	Verification required	Reversibility	Main cost
Parallelizing independent calls	Serial waits	R0	Latency benchmark	Code change	Low effort
Precomputation	Online compute	R2	Offline metrics, staleness slices	Pipeline change	Batch jobs, storage
Caching	Repeated work	R2	Hit rate, staleness, slice parity	Flag	Memory, invalidation
Candidate budget	Work in every later stage	R1 to R3	Offline metrics; online test for large cuts	Configuration	Quality headroom
Search-time parameters	Retrieval compute	R1	Recall and end-to-end metric	Configuration	Low effort
Index compression	Memory traffic	R1	Recall and end-to-end metric	Re-index	Build time
Intra-request batching	Per-call overhead	R0	Score parity	Code change	Low effort
Dynamic batching	Per-call overhead at load	R0	Latency at several loads	Configuration	Added waiting time
Export and graph optimization	Operator overhead	R0	Score parity	Artifact swap	Export pipeline
Quantization	Compute and memory	R1	Score, rank, slice parity	Artifact swap	Hardware dependent
Distillation	Model compute	R2	Full ladder, shadow comparison	Retrain	Training effort
Pruning, embedding compression	Compute and memory	R2	Full ladder	Retrain	Training effort
Hardware change	Compute time	R0	Score parity, cost check	Infrastructure	Running cost

Lever	Time removed	Risk class	Verification required	Reversibility	Main cost
Hedging	Tail of slow replicas	R0	Added load, tail at load	Configuration	Extra requests

14.10.2 Measure First, Then Optimize

Knuth's remark about premature optimization is often quoted as an argument against optimizing at all. In full, it argues for measurement: "We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%. A good programmer will not be lulled into complacency by such reasoning, he will be wise to look carefully at the critical code; but only after that code has been identified" (Knuth, 1974).

The critical fraction of a recommender is the set of stages that dominate slow requests, and the evidence in earlier sections shows why it cannot be identified from rules of thumb. Whether quantization speeds up a model depends on the processor and can be negative (ONNX Runtime, 2026b). The best batch size lies in a middle range that differs across systems (Zeng, 2020). The request size at which an accelerator beats a CPU depends on the model and the tail target (Gupta et al., 2020b). The size a cache needs depends on a popularity exponent that varies between workloads (Breslau et al., 1999). Each is a measurement the team has to make.

A few rules do hold without measurement, because they follow from arithmetic or from the failure modes already described. Aggregate histograms and never average percentiles. Generate load open-loop, or correct for coordinated omission. Warm up before measuring. Set and propagate deadlines, and bound retries with budgets and jitter. Score all of a request's candidates in batched calls. Leave stages that are off the critical path of slow requests alone. Verify quality after every approximation, and exercise fallback paths regularly.

14.11 Proving the Faster System Is the Same System

Every lever with a risk class above R0 changes what the recommender returns, and even R0 levers can change it by accident: an export that drops a preprocessing step, a cache keyed on too little, a batched path that pads inputs differently. A speedup is finished only when the team has shown that the faster system still recommends what the slower one did, or has measured how and where it differs. Figure 14.12 shows that verification as a ladder of increasingly expensive checks.



Figure 14.12: A verification ladder for an optimized serving path. Each step compares the optimized path with the reference on the same logged requests: first raw scores, then rankings, then offline quality metrics, then the same checks on vulnerable slices, and finally latency at fixed load with intervals. Verification stops at the first step that fails, and each step costs less than the next; the online release gates and experiments come after the ladder.

14.11.1 Replay Sets and Score Parity

The foundation is a replay set: a sample of logged production requests, each stored with the inputs the pipeline saw and the reference outputs of the current system. Running both the reference and the optimized path on the same replay set isolates the change from everything else that varies in production. The sample should cover the traffic's variety, including long requests and filtered queries, and it should be refreshed as traffic changes.

Score parity compares raw model outputs request by request: the maximum and mean absolute difference between reference and optimized scores, and their Pearson and Spearman correlations. For R0 levers, outputs should be identical up to floating-point rounding, and anything larger points to a bug. For quantization, larger differences are expected, and the question becomes whether they matter. When they do, ONNX Runtime provides debugging tools that match intermediate tensors between a full-precision model and its quantized version, so the team can find the layer where the two diverge (ONNX Runtime, 2026b).

14.11.2 Rank and Metric Parity

Scores can shift while rankings stay the same, since a uniform shift changes no order, and small score differences can swap neighbors that users would see. Rank parity compares the orderings per request. Top-K overlap counts the share of the reference's top K items that the optimized path also places in its top K. Kendall's tau measures agreement over all pairs of the candidates both paths scored; when the candidate sets differ, top-K overlap and rank-biased overlap still apply. Rank-biased overlap, proposed by Webber et al. (2010), weights agreement at the top of the list more heavily than agreement further down, which suits recommendation, where users see only the first few slots. For two rankings S and T, let A_d be the fraction of items shared by their top d positions. Then

$$\text{RBO}(S, T, p) = (1 - p) \sum_{d=1}^{\infty} p^{d-1} A_d,$$

where the parameter p between 0 and 1 sets how steeply weight falls with depth. Real lists stop at some depth k, and cutting the sum there understates agreement: two identical lists of length 10 score only about 0.65 at $p = 0.9$. Webber et al. (2010) therefore extrapolate, assuming that the agreement seen at depth k continues below it:

$$\text{RBO}_{\text{ext}} = A_k p^k + \frac{1 - p}{p} \sum_{d=1}^{k} A_d \, p^d,$$

which equals 1 for identical lists, so the truncated sum is only a lower bound on it.

Metric parity asks whether the offline metrics the team uses to judge models, such as NDCG, hit rate, or AUC, move beyond their run-to-run noise. It also checks calibration, because a model whose scores feed a value function (Chapter 8) can keep its ranking while its predicted probabilities drift, and the mean predicted probability compared with the observed rate shows that drift directly.

14.11.3 Slice Parity

Aggregate parity can hide damage concentrated in small groups, and latency optimizations often concentrate it. Caches miss most on new users and new items. Distilled and pruned models lose most where the teacher's knowledge was thinnest. Candidate cuts and approximate search lose long-tail items first. Filtered queries take different code paths. Users with long histories cost the most to serve, which makes them the first target of truncation. Slice parity repeats the score, rank, and metric checks for each such group: cold-start users, long-tail items, filtered queries, and long-history users. Instagram's distilled first-pass model builds agreement with the later stages into its training objective (Medvedev, 2019), and that design still leaves slices to check, because a student can match its teacher on average while diverging where data is scarce.

14.11.4 The Performance Check and the Handoff

The last step confirms that the change delivered the speedup it promised, with the methodology of Section 14.3: a fixed benchmark at the load level in the latency target, warmed up, repeated, and reported with intervals for the reference and the optimized path. Kept in the repository and run on every change to the serving path, the same benchmark becomes a performance regression test, so a later change that quietly gives back the gain is caught before release.

Passing the ladder means the change is ready for the release gates of Chapter 15 (Section 15.9.5): shadow deployment, canary, and champion-challenger comparison on live traffic. DoorDash's prediction service shows the shadow mechanism in practice. It ran shadow predictions asynchronously, so test models scored live requests without delaying the official responses (Zeng, 2020). For R3 levers, whose effect depends on how users respond, an online experiment with latency tracked as a guardrail metric is the final check (Kohavi et al., 2020), using the methods of Chapter 16. Exercise 2 Part B (Section 14.13) produces a parity report of this kind for each variant, overall and on four slices.

14.12 Summary

Latency matters to a recommender for two reasons. Slowdown experiments in web search showed that delays too small to notice still reduce use and revenue, and in recommendation the time a request allows decides which models can run at all. Because public evidence specific to recommenders is scarce and every system differs, this chapter treated latency work as a loop of measurement and verification.

The loop has four parts. Measure: define a latency target with a percentile, threshold, measurement point, load level, and window; record histograms; warm up; repeat runs with intervals; and generate load open-loop so that stalls are not hidden by coordinated omission. Attribute: trace requests, find the critical path, and study the requests above the target percentile, splitting each stage's time into service and waiting, with the residual as a check on the instrumentation. Choose: estimate each lever's gain from its stage's share of tail time, probe its real gain cheaply, classify its quality risk, weigh its cost and reversibility, apply one change, and measure again, because the bottleneck moves. Verify: climb the ladder from score parity through rank, metric, and slice parity to a performance benchmark before any change reaches users.

Along the way, the chapter explained why tails grow with utilization, variability, and fan-out, and surveyed the levers: precomputing and caching, trimming candidates, tuning approximate search, batching and compressing models, and keeping every request within its deadline.

Apart from the few rules of Section 14.10.2 that follow from arithmetic, every optimization is a hypothesis to test on the team's own system. Chapter 15 takes the fast, verified service into production and keeps its latency target true with monitoring, error budgets, and release gates, and Chapter 16 provides the experimental methods to value each change with real users.

14.13 Exercises

Each exercise has two parts, run from the companion code on a CPU with real data when it is available and synthetic data otherwise; its README gives setup and the main flags. The exercises say what to look for, leaving the results to you.

Exercise 1: Where the Tail Comes From

Part A, `python -m scripts.ex1a_queueing_sim`, simulates queues in virtual time, so its results do not depend on the machine. It injects a server stall and records the tail from closed-loop clients, from an open-loop schedule timed from intended send times, and from closed-loop records corrected for the expected interval; explain the gap using Section 14.3.3. It then checks simulated single-server queues and fan-out against Sections 14.5.1 and 14.5.2, repeats the queues with bursty arrivals and bimodal service times, and hedges after the p95 delay, and after the p50 for contrast, at several utilizations. Record the backup requests and tail change at each utilization. Note whether total server work rises or falls, and which assumption about where slowness comes from decides it.

Part B, `python -m scripts.ex1b_pipeline_attribution --load 0.3,0.5,0.7,0.85`, runs a seven-stage request on a thread pool: admission (S1), request context (S2), two-tower retrieval over a FAISS index plus a popularity branch (S3), a fetch from a simulated remote feature store (S4), an XGBoost light ranker and Chapter 7's DCN-V2 as the heavy ranker (S5), MMR re-ranking (S6), and serialization (S7). Item vectors come from Chapter 5's Coveo SIGIR eCom 2021 data or a synthetic catalog. An open-loop Poisson generator drives the pipeline at fractions of its measured capacity, and the report sets the critical-path composition of requests above the p99, split into service and waiting

time, beside that of requests near the median, with the residual R. Each `--lever` option, such as `--lever feature-cache=on` or `--lever candidates=200`, is compared with the baseline on its own. Before running a lever, write down its estimate from the stage's tail share (Section 14.10.1), then compare it with the measured p99 change and its paired interval across repeats. Identify any lever that speeds up its own stage without moving the end-to-end tail, and explain why from the critical path. At the highest load, check whether any gain exceeds its estimate, and why.

Exercise 2: Faster Retrieval and Ranking With the Same Results

Part A, `python -m scripts.ex2a_ann_operating_points --recall-floor 0.95`, sweeps nprobe and `efSearch` for exact search, IVF, HNSW, and compressed IVF over the Coveo product embeddings that Chapter 5 searches, with queries built from browsing sessions, and over a larger synthetic catalog, at K of 100, 500, and 1,000. Each setting reports recall@K against exact search, next-item hit rate on the Coveo sessions, and latency percentiles with intervals. It marks dominated settings and selects the fastest one that meets the floor, exact search included. Look for whether hit rate falls with recall, whether frontiers cross, and whether the chosen configuration changes with K.

Part B, `python -m scripts.ex2b_ranker_inference_parity`, trains Chapter 7's DCN-V2 on a short window of the Yambda listening data used in Chapters 6 and 7, or on synthetic data with `--data synthetic`. It exports the model to ONNX and benchmarks eager PyTorch against ONNX Runtime at the basic, extended, and all optimization levels and with dynamic int8 quantization, at batch sizes from 1 to 1,024 and at one thread and all threads. Each variant is checked against the PyTorch reference for score parity, rank parity (top-K overlap, Kendall's tau, and rank-biased overlap), AUC change, and calibration, overall and for short- and long-history users and popular and tail items, and it passes only if its AUC, top-10 overlap, and calibration tolerances hold on every slice. Look for where the batch-size curve bends, whether quantization or more threads help on your processor, and whether a faster variant fails parity on a slice.

References

Amatriain, X., and Basilico, J. (2013). "System Architectures for Personalization and Recommendation." Netflix Technology Blog. https://netflixtechblog.com/system-architectures-for-personalization-and-recommendation-e081aa94b5d8

Amdahl, G. M. (1967). "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities." Proceedings of the AFIPS Spring Joint Computer Conference, 483-485. ACM. https://doi.org/10.1145/1465482.1465560

Arapakis, I., Bai, X., and Cambazoglu, B. B. (2014). "Impact of Response Latency on User Behavior in Web Search." Proceedings of the 37th International ACM SIGIR Conference on Research and Development in Information Retrieval, 103-112. ACM. https://doi.org/10.1145/2600428.2609627

Aumüller, M., Bernhardsson, E., and Faithfull, A. (2020). "ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms." Information Systems 87, 101374. https://doi.org/10.1016/j.is.2019.02.006

Bernardi, L., Mavridis, T., and Estevez, P. (2019). "150 Successful Machine Learning Models: 6 Lessons Learned at Booking.com." Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1743-1751. ACM. https://doi.org/10.1145/3292500.3330744

Beyer, B., Jones, C., Petoff, J., and Murphy, N. R. (Eds.). (2016). Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media. https://sre.google/sre-book/table-of-contents/

Breslau, L., Cao, P., Fan, L., Phillips, G., and Shenker, S. (1999). "Web Caching and Zipf-like Distributions: Evidence and Implications." Proceedings of IEEE INFOCOM 1999. IEEE. https://doi.org/10.1109/INFCOM.1999.749260

Brooker, M. (2019). "Timeouts, Retries, and Backoff with Jitter." Amazon Builders' Library. https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/

Brutlag, J. (2009). "Speed Matters for Google Web Search." Google. https://services.google.com/fh/files/blogs/google_delayexp.pdf

Correa, M., Gonigberg, A., and West, D. (2020). "Keeping Netflix Reliable Using Prioritized Load Shedding." Netflix Technology Blog. https://netflixtechblog.com/keeping-netflix-reliable-using-prioritized-load-shedding-6cc827b02f94

Crankshaw, D., Wang, X., Zhou, G., Franklin, M. J., Gonzalez, J. E., and Stoica, I. (2017). "Clipper: A Low-Latency Online Prediction Serving System." 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17). USENIX Association. https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/crankshaw

Dean, J., and Barroso, L. A. (2013). "The Tail at Scale." Communications of the ACM 56(2), 74-80. https://doi.org/10.1145/2408776.2408794

Deng, W., Pan, J., Zhou, T., Kong, D., Flores, A., and Lin, G. (2021). "DeepLight: Deep Lightweight Feature Interactions for Accelerating CTR Predictions in Ad Serving." Proceedings of the 14th ACM International Conference on Web Search and Data Mining. ACM. https://doi.org/10.1145/3437963.3441727

Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., and Jégou, H. (2024). "The Faiss Library." arXiv:2401.08281. https://arxiv.org/abs/2401.08281

Eksombatchai, C., Jindal, P., Liu, J. Z., Liu, Y., Sharma, R., Sugnet, C., Ulrich, M., and Leskovec, J. (2018). "Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time." Proceedings of the 2018 World Wide Web Conference. ACM. https://doi.org/10.1145/3178876.3186183

Eksombatchai, P., and Zhang, Z. (2022). "GPU-accelerated ML Inference at Pinterest." Pinterest Engineering Blog. https://medium.com/pinterest-engineering/gpu-accelerated-ml-inference-at-pinterest-ad1b6a03a16d

FastAPI. (2026). "Concurrency and async / await." FastAPI Documentation. Accessed 8 November 2026. https://fastapi.tiangolo.com/async/

Georges, A., Buytaert, D., and Eeckhout, L. (2007). "Statistically Rigorous Java Performance Evaluation." Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA), 57-76. ACM. https://doi.org/10.1145/1297027.1297033

Gregg, B. (2012). "Thinking Methodically about Performance." ACM Queue 10(12). https://queue.acm.org/detail.cfm?id=2413037

Guan, H., Malevich, A., Yang, J., Park, J., and Yuen, H. (2019). "Post-Training 4-bit Quantization on Embedding Tables." NeurIPS 2019 Workshop on Systems for Machine Learning. arXiv:1911.02079. https://arxiv.org/abs/1911.02079

Gupta, U., Wu, C.-J., Wang, X., Naumov, M., Reagen, B., Brooks, D., Cottel, B., Hazelwood, K., Hempstead, M., Jia, B., Lee, H.-H. S., Malevich, A., Mudigere, D., Smelyanskiy, M., Xiong, L., and Zhang, X. (2020a). "The Architectural Implications of Facebook's DNN-based Personalized Recommendation." 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA). IEEE. https://doi.org/10.1109/HPCA47549.2020.00047

Gupta, U., Hsia, S., Saraph, V., Wang, X., Reagen, B., Wei, G.-Y., Lee, H.-H. S., Brooks, D., and Wu, C.-J. (2020b). "DeepRecSys: A System for Optimizing End-to-End At-Scale Neural Recommendation Inference." 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA). IEEE. https://doi.org/10.1109/ISCA45697.2020.00084

Harchol-Balter, M. (2013). Performance Modeling and Design of Computer Systems: Queueing Theory in Action. Cambridge University Press. https://doi.org/10.1017/CBO9781139226424

Hinton, G., Vinyals, O., and Dean, J. (2015). "Distilling the Knowledge in a Neural Network." arXiv:1503.02531. https://arxiv.org/abs/1503.02531

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. (2018). "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference." Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2704-2713. IEEE. https://doi.org/10.1109/CVPR.2018.00286

Knuth, D. E. (1974). "Structured Programming with go to Statements." ACM Computing Surveys 6(4), 261-301. https://doi.org/10.1145/356635.356640

Kohavi, R., Deng, A., Frasca, B., Walker, T., Xu, Y., and Pohlmann, N. (2013). "Online Controlled Experiments at Large Scale." Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1168-1176. ACM. https://doi.org/10.1145/2487575.2488217

Kohavi, R., Tang, D., and Xu, Y. (2020). Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing. Cambridge University Press. Chapter 5, "Speed Matters: An End-to-End Case Study," 81-89. https://doi.org/10.1017/9781108653985.008

Kusupati, A., Bhatt, G., Rege, A., Wallingford, M., Sinha, A., Ramanujan, V., Howard-Snyder, W., Chen, K., Kakade, S., Jain, P., and Farhadi, A. (2022). "Matryoshka Representation Learning." Advances in Neural Information Processing Systems 35 (NeurIPS 2022). https://proceedings.neurips.cc/paper_files/paper/2022/hash/c32319f4868da7613d78af9993100e42-Abstract-Conference.html

Little, J. D. C. (1961). "A Proof for the Queuing Formula: L = λW." Operations Research 9(3), 383-387. https://doi.org/10.1287/opre.9.3.383

Locust. (2026). "Writing a locustfile." Locust Documentation. Accessed 8 November 2026. https://docs.locust.io/en/stable/writing-a-locustfile.html

Lui, M., Yetim, Y., Özkan, Ö., Zhao, Z., Tsai, S.-Y., Wu, C.-J., and Hempstead, M. (2021). "Understanding Capacity-Driven Scale-Out Neural Recommendation Inference." 2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS). IEEE. https://arxiv.org/abs/2011.02084

Medvedev, I. (2019). "Powered by AI: Instagram's Explore Recommender System." Meta AI Blog. https://ai.meta.com/blog/powered-by-ai-instagrams-explore-recommender-system/

Netflix Technology Blog. (2018). "Evolution of Application Data Caching: From RAM to SSD." Netflix Technology Blog. https://netflixtechblog.com/evolution-of-application-data-caching-from-ram-to-ssd-a33d6fa7a690

Olston, C., Fiedel, N., Gorovoy, K., Harmsen, J., Lao, L., Li, F., Rajashekhar, V., Ramesh, S., and Soyke, J. (2017). "TensorFlow-Serving: Flexible, High-Performance ML Serving." NIPS 2017 Workshop on ML Systems. arXiv:1712.06139. https://arxiv.org/abs/1712.06139

ONNX Runtime. (2026a). "Graph Optimizations in ONNX Runtime." ONNX Runtime Documentation. Accessed 8 November 2026. https://onnxruntime.ai/docs/performance/model-optimizations/graph-optimizations.html

ONNX Runtime. (2026b). "Quantize ONNX Models." ONNX Runtime Documentation. Accessed 8 November 2026. https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html

OpenTelemetry. (2026). "Traces." OpenTelemetry Documentation. Accessed 8 November 2026. https://opentelemetry.io/docs/concepts/signals/traces/

Patel, L., Kraft, P., Guestrin, C., and Zaharia, M. (2024). "ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data." Proceedings of the ACM on Management of Data 2(3), Article 120. ACM. https://doi.org/10.1145/3654923

Pinterest Engineering. (2025). "A Decade of AI Platform at Pinterest." Pinterest Engineering. https://medium.com/pinterest-engineering/a-decade-of-ai-platform-at-pinterest-4e3b37c0f758

Shi, H.-J. M., Mudigere, D., Naumov, M., and Yang, J. (2020). "Compositional Embeddings Using Complementary Partitions for Memory-Efficient Recommendation Systems." Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. ACM. https://doi.org/10.1145/3394486.3403059

Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., and Shanbhag, C. (2010). "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure." Google Technical Report dapper-2010-1. https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/

Sumbaly, R., Kreps, J., Gao, L., Feinberg, A., Soman, C., and Shah, S. (2012). "Serving Large-scale Batch Computed Data with Project Voldemort." 10th USENIX Conference on File and Storage Technologies (FAST 12). USENIX Association. https://www.usenix.org/conference/fast12/serving-large-scale-batch-computed-data-project-voldemort

Tang, J., and Wang, K. (2018). "Ranking Distillation: Learning Compact Ranking Models With High Performance for Recommender System." Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2289-2298. ACM. https://doi.org/10.1145/3219819.3220021

Tene, G. (2012). "How NOT to Measure Latency." QCon San Francisco 2012. https://qconsf.com/sf2012/dl/qcon-sanfran-2012/slides/GilTene_HowNotToMeasureLatency.pdf

Walecka, S., and Gallatin, K. (2023). "Improving Support for Deep Learning in Etsy's ML Platform." Code as Craft, Etsy Engineering. https://www.etsy.com/codeascraft/improving-support-for-deep-learning-in-etsy39s-ml-platform

Webber, W., Moffat, A., and Zobel, J. (2010). "A Similarity Measure for Indefinite Rankings." ACM Transactions on Information Systems 28(4), Article 20. https://doi.org/10.1145/1852102.1852106

Zeng, C. (2020). "Meet Sibyl: DoorDash's New Prediction Service, Learn about its Ideation, Implementation and Rollout." DoorDash Engineering. https://careersatdoordash.com/blog/doordashs-new-prediction-service/